

SNOWFLAKE ASSIGNMENT

AIM:

A retail company receives daily order files from its sales application.

The source application uploads CSV files into an Azure Storage Account.

Snowflake should automatically load the new files whenever they arrive in Azure Storage.

The company wants:

1. A separate Snowflake warehouse for loading data
2. A separate user and role for data engineering
3. Data ingestion from Azure Storage Account
4. Initial load from CSV file
5. Automatic incremental loading when new CSV files are uploaded
6. Avoid duplicate records in the final table

Important Concept for Students

Snowflake does not automatically detect new rows added inside the same old file.

Best practice is:

Do not edit the already uploaded CSV file.

Instead, upload a new CSV file for every new batch.

Example:

retail_orders_initial.csv

retail_orders_incremental_20260508.csv

retail_orders_incremental_20260509.csv

retail_orders_incremental_20260510.csv

Snowpipe detects and loads new files, not newly edited rows inside the same file.

PART A--- Snowflake Setup

Create a database named RETAIL_DB.

Create a schema named RETAIL_DB.ORDERS_DEMO.

Create a warehouse named RETAIL_LOAD_WH.

Create a role named RETAIL_DATA_ENGINEER_ROLE.

Create a user named RETAIL_ENGINEER_USER.

Grant required permissions to the role and user.

```

-----PART A-----
-----warehouse creation-----
create or replace warehouse retail_load_wh
with
warehouse_size = 'xsmall'
initially_suspended = true
auto_suspend = 60
auto_resume = true;
-----database creation-----
create or replace database retail_db;
-----schema creation-----
create or replace schema orders_demo;
-----roles creation -----
create role retail_data_engineer_role;
-----user creation-----
CREATE USER retail_engineer_user PASSWORD = 'reu@123' LOGIN_NAME = 'reu_user' DEFAULT_ROLE='retail_data_engineer_role' DEFAULT_WAREHOUSE =
'retail_load_wh' MUST_CHANGE_PASSWORD = FALSE;
-----grant permissions-----
grant role retail_data_engineer_role to user retail_engineer_user;
grant usage on warehouse retail_load_wh to role
retail_data_engineer_role;
grant usage on database retail_db to role
retail_data_engineer_role;

```

PART B -----Azure Storage Setup

Create an Azure Storage Account.

The screenshot shows the Azure portal interface for the 'retaildatakeerthi' Storage account. The left sidebar contains navigation options like 'Create a resource', 'Home', 'Dashboard', 'All services', and 'FAVORITES'. The main content area shows the 'Overview' tab with a search bar and a list of actions (Upload, Open in Explorer, Delete, Move, Refresh, Open in mobile, CLI / PS). The 'Essentials' section on the right lists key account details:

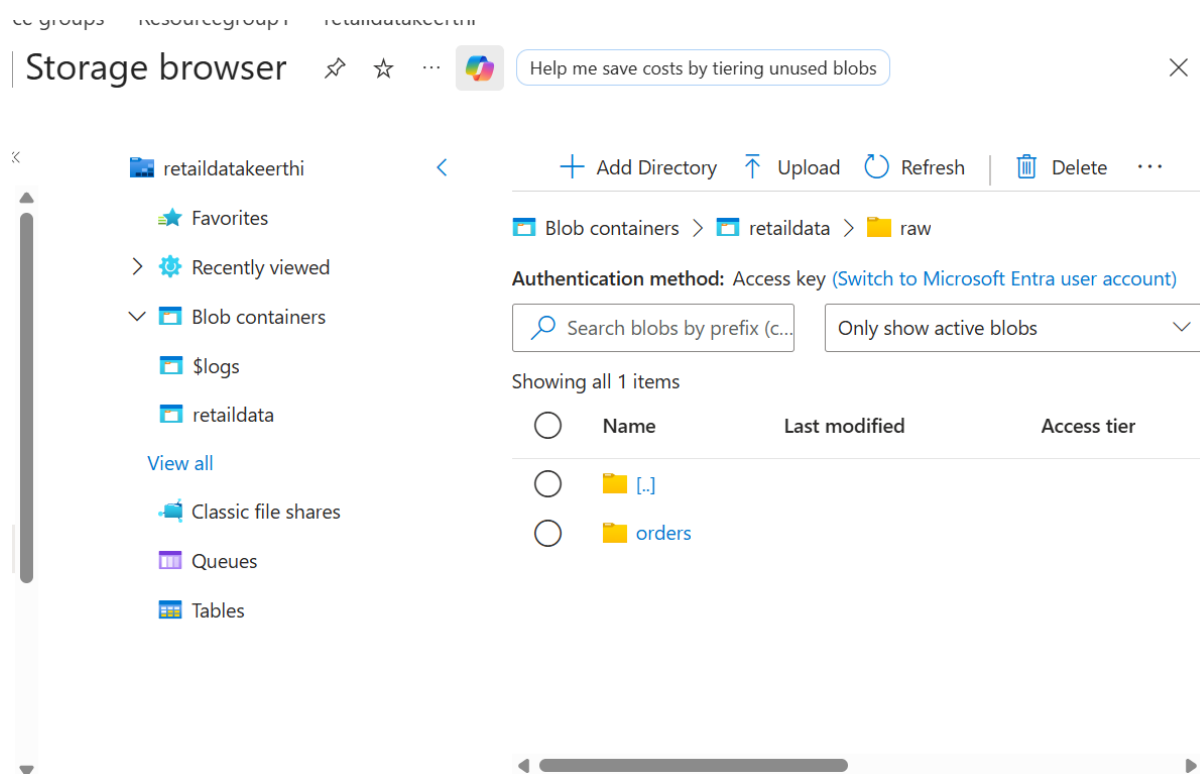
Property	Value
Resource group	Resourcegroup1
Location	southindia
Subscription	Azure subscription 1
Subscription ID	da9963c3-5720-4393-80e2-f9d0cd0feac5
Disk state	Available
Performance	Standard
Replication	Locally redundant storage (LRS)
Account kind	StorageV2 (general purpose v2)
Provisioning state	Succeeded
Created	5/21/2026, 3:36:12 PM

Create a container named retaildata.

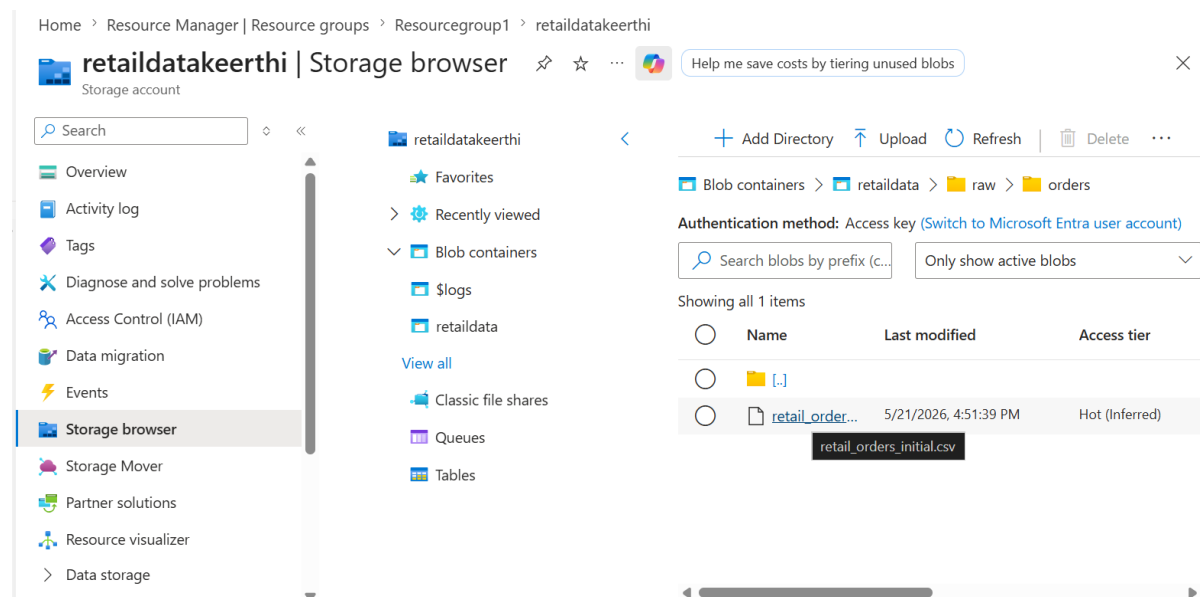
The screenshot shows the 'Storage browser' view for the 'retaildatakeerthi' Storage account. The left sidebar shows navigation options like 'Overview', 'Activity log', 'Tags', 'Diagnose and solve problems', 'Access Control (IAM)', 'Data migration', 'Events', 'Storage browser', 'Storage Mover', 'Partner solutions', 'Resource visualizer', and 'Data storage'. The main content area shows the 'Blob containers' section with a search bar and a list of containers. The 'retaildata' container is highlighted, and its details are shown on the right:

Name	Last modified	Anonymous
\$logs	5/21/2026, 3:36:43 PM	Private
retaildata	5/21/2026, 3:51:34 PM	Private

Inside the container, create this folder path:raw/orders/

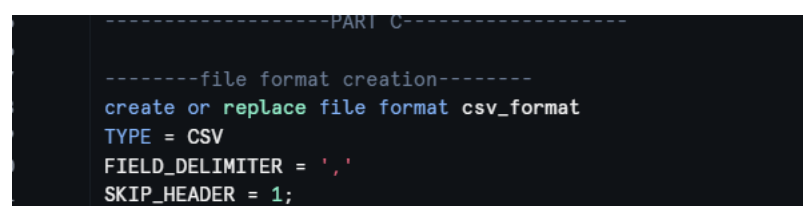


Upload the initial CSV file: retail_orders_initial.csv



PART C ----- Snowflake External Stage

Create a file format for CSV.



Create an Azure storage integration or external stage.

```
-----azure storage integration-----
create or replace storage integration azure_storage
type = external_stage
storage_provider = azure
enabled = true
azure_tenant_id = '24b23ea2-47d3-4211-85f7-b54c8fa30f11'
storage_allowed_locations = ('azure://retaildatakeerthi.blob.core.windows.net/retaildata/raw/orders/')

desc storage integration azure_storage;
```

Create an external stage pointing to:

azure://<storage_account>.blob.core.windows.net/retail-data/raw/orders/

```
-----stage creation-----
create or replace stage retail_stage
storage_integration = azure_storage
url='azure://retaildatakeerthi.blob.core.windows.net/retaildata/raw/orders/'
file_format = csv_format
```

List files from the stage.

```
-----listing stage-----
list @retail_stage
```

PART D-----Initial Data Load

Create a raw landing table named ORDERS_RAW.

```
-----table creation -----
CREATE OR REPLACE TABLE orders_raw
(
  order_id varchar(10),
  order_date date,
  customer_id varchar(10),
  customer_name string,
  product_id varchar(10),
  product_name string,
  category string,
  quantity number,
  unit_price decimal,
  total_amount decimal,
  status string,
  region string
);

copy into orders_raw
from @retail_stage
file_format = csv_format;

select * from orders_raw;
```

Create a final table named ORDERS_FINAL.

Load the initial file into ORDERS_RAW.

Insert data into ORDERS_FINAL.

```
-----final table creation -----
```

```
CREATE OR REPLACE TABLE orders_final  
as select * from orders_raw;  
  
select * from orders_final;
```

PART E --- Incremental Loading

Create a Snowpipe object to load new files automatically into ORDERS_RAW.

```
-----PART E-----
```

```
create or replace notification integration notify_queue  
type=queue  
notification_provider=azure_storage_queue  
enabled = true  
azure_tenant_id = '24b23ea2-47d3-4211-85f7-b54c8fa30f11'  
azure_storage_queue_primary_uri='https://retaildatakeerthi.queue.core.windows.net/azure-queue'
```

```
desc notification integration notify_queue;
```

```
-----azure pipe-----
```

```
create or replace pipe azure_pipe  
auto_ingest = true  
integration = notify_queue  
as  
copy into orders_raw  
from @retail_stage;  
  
drop pipe azure_pipe;  
  
SELECT SYSTEM$PIPE_STATUS( 'azure_pipe' );
```

Create a stream on ORDERS_RAW.

```
-----stream creation-----
```

```
create or replace stream orders_stream on table orders_raw;
```

Create a task to merge new rows from the stream into ORDERS_FINAL.

```
-----task creation-----
```

```
CREATE OR REPLACE TASK orders_task  
WAREHOUSE = 'retail_load_wh'  
SCHEDULE = '1 MINUTE'  
AS  
MERGE INTO orders_final f  
USING  
(  
    SELECT  
        order_id,  
        order_date,  
        customer_id,  
        customer_name,  
        product_id,  
        product_name,  
        category,  
        quantity,  
        unit_price,  
        total_amount,  
        status,  
        region  
    FROM orders_stream  
) s  
ON f.order_id = s.order_id
```

```

WHEN MATCHED THEN
UPDATE SET
    f.order_date = s.order_date,
    f.customer_id = s.customer_id,
    f.customer_name = s.customer_name,
    f.product_id = s.product_id,
    f.product_name = s.product_name,
    f.category = s.category,
    f.quantity = s.quantity,
    f.unit_price = s.unit_price,
    f.total_amount = s.total_amount,
    f.status = s.status,
    f.region = s.region

```

```

WHEN NOT MATCHED THEN
INSERT

```

```

(
    order_id,
    order_date,
    customer_id,
    customer_name,
    product_id,
    product_name,
    category,
    quantity,
    unit_price,
    total_amount,
    status,
    region
)

```

```

VALUES

```

```

(
    s.order_id,
    s.order_date,
    s.customer_id,
    s.customer_name,
    s.product_id,
    s.product_name,
    s.category,

```

```

VALUES

```

```

(
    s.order_id,
    s.order_date,
    s.customer_id,
    s.customer_name,
    s.product_id,
    s.product_name,
    s.category,
    s.quantity,
    s.unit_price,
    s.total_amount,
    s.status,
    s.region
);

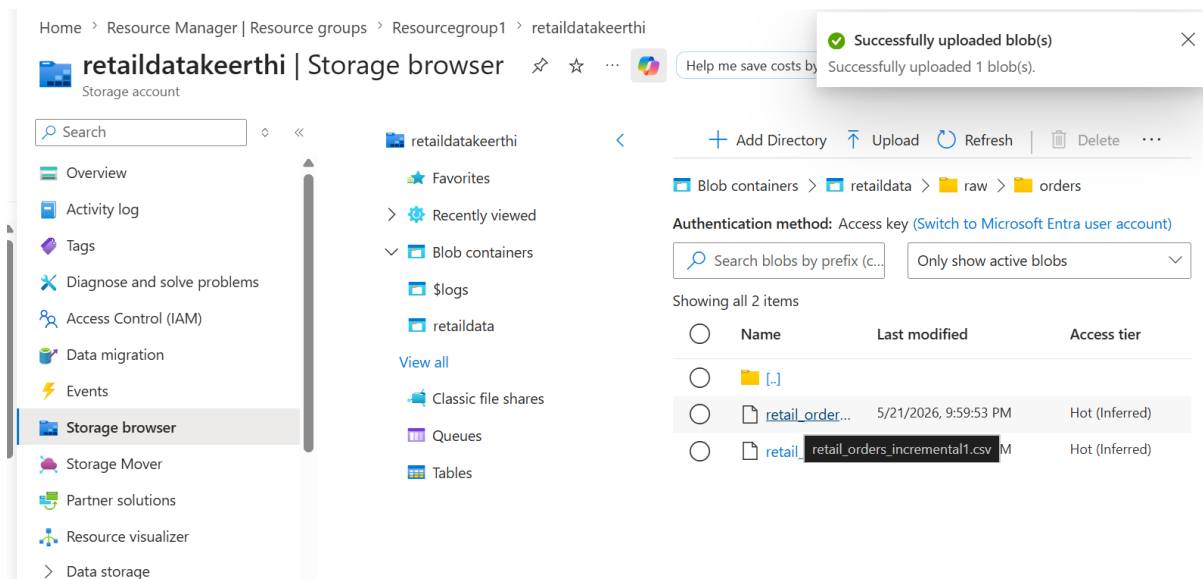
```

```

alter task orders_task resume

```

Upload another CSV file into Azure Storage.



Verify that new rows are automatically loaded into Snowflake.

BEFORE ADDING FILE TO BLOB(INTIALIZING PIPE AND TASK) 30 ROWS IN ORDERS_RAW

The screenshot shows a Snowflake query result for the command `select count(*) from orders_raw;`. The result is displayed in a table with one row and one column, showing a count of 30.

COUNT(*)
30

BEFORE INITIALIZING PIPE AND TASK 30 ROWS IN ORDERS_FINAL TABLE

The screenshot shows a Snowflake query result for the command `select count(*) from orders_final;`. The result is displayed in a table with one row and one column, showing a count of 30.

COUNT(*)
30

AFTER INITIALIZING PIPE DATA OF NEW FILE ADDED ORDERS_RAW

The screenshot shows a Snowflake query result for the command `select count(*) from orders_raw;`. The result is displayed in a table with one row and one column, showing a count of 40.

COUNT(*)
40

STREAM DATA AFTER PIPE TRNFER NEWLY ADDED FILE DATA TO ORDERS_RAW(10 INSERTS)

Results (just now)

Table Chart Pivot

10 rows 240ms

	ORDER_ID	ORDER_DATE	CUSTOMER_ID	CUSTOMER_NAME	PRODUCT_ID	PRODUCT_NAME	CATEGORY	QUANTITY	UNIT_PRICE
	ORD01300	2026-05-08	C105	Vikram Nair	P005	Sunglasses	Accessories	1	10
	ORD01301	2026-05-08	C102	Priya Reddy	P002	Running Shoes	Footwear	2	20
	ORD01302	2026-05-08	C107	Karthik Rao	P003	Coffee Maker	Appliances	1	14
	ORD01303	2026-05-08	C104	Sneha Iyer	P007	Water Bottle	Sports	5	42
	ORD01304	2026-05-08	C102	Priya Reddy	P010	Blender	Appliances	5	40
	ORD01305	2026-05-08	C106	Divya Pillai	P010	Blender	Appliances	5	20
	ORD01306	2026-05-08	C103	Rahul Mehta	P008	Desk Lamp	Home	4	18
	ORD01307	2026-05-08	C108	Ananya Singh	P007	Water Bottle	Sports	4	28
	ORD01308	2026-05-08	C106	Divya Pillai	P008	Desk Lamp	Home	1	18
	ORD01309	2026-05-08	C110	Meena Joshi	P002	Running Shoes	Footwear	2	20

AFTER INTIALIZING TASK DATA IN STREAM ADDED ORDERS_FINAL

```
190 | select count(*) from orders_final;
```

Results (just now)

Table Chart Pivot

1 row 45ms

#	COUNT(*)
1	40

STREAM BECOME EMPTY AFTER PERFORMING DML ON TABLE ORDERS_FINAL

```
111 | select * from orders_stream;
```

Results (just now)

Table Chart Pivot

0 rows 99ms

ORDER_ID	ORDER_DATE	CUSTOMER_ID	CUSTOMER_NAME	PRODUCT_ID	PRODUCT_NAME	CATEGORY	QUANTITY	UNIT_P
----------	------------	-------------	---------------	------------	--------------	----------	----------	--------

Query produced no results